

```
ans.make_table(lambda(s,r): (s.dst, s.dport, r.sprintf("%iP.%i%  
%t %iP.%i%t %t %TCP.flags%")))
```

A sample usage of this script:

```
root@ubiserv:~# python SYN_Flood_Scapy.py 192.168.56.102
```

Finally, the script reports SA (SYN-ACK) responses, and gives the results as answered/unanswered packets. The target's reply of SA shows that it 'thinks' the ACK from attacker/initiator was lost; hence, it keeps re-sending it, for an interval specified by the kernel. The connection, on the target server, remains in the SYN_RECV condition for 3 minutes for each port, as per the *net.ipv4.tcp_synack_retries* parameter, which is set to 5 in Linux. After these retries, the kernel closes the connection.

Here is the seed of a SYN flood. Millions of unanswered SYN requests to the target server can fill the buffer up completely, leaving it unable to serve legitimate clients. Now let's look into custom prevention methods.

Prevention measures on target server

As a prevention measure, I created a shell script to generate *iptables* rules, which will be purged automatically on exit from the script. This solution will reject all suspicious TCP connections with a TCP RST packet, to prevent potential DDOS. All connections in the SYN_RECV state will be closed forcibly with RST packets. I allow 25 attempts from a single IP address, to take care of packet loss, which sometimes does happen, due to network errors. Thus, legitimate clients have a chance to reconnect. After 25 attempts from the same IP address, SYN packets from that IP address will be rejected as intentional flooding (an *iptables* rule entry is added for malicious IP addresses, to REJECT with a TCP RST flag), and it will be logged for tracking purposes. The time interval at which to check for connections has to be specified (the script will *sleep* for the given time interval). If the server has a significant load, lower intervals are better, for more frequent scans. This will also log the number of connections made from each IP address, for further analysis. The script can be terminated with the EXIT signal. It uses a while loop with the *continue* jump statement, so that existing IP address rules will not be re-entered.

The *SYN_Flood_Prevention.sh* script is as follows:

```
# Description : SYN Flood Prevention using iptables against Scapy  
SYN packets generated  
> /var/log/DDOS_IP.log  
> /tmp/test1.txt  
> /tmp/test2.txt  
trap "echo ;echo Caught EXIT signal;iptables -F;echo Iptables  
entries cleared;echo HAXOR SVP" EXIT  
while true;
```

```
# root@ubiserv:~# ./SYN_Flood_Scapy.py 192.168.56.102  
WARNING: No route found for IPw destination : [no default route?]  
Field Values of packet sent  
version 1 ByteField = 4 (4)  
len 1 ByteField = 60 (60)  
len 1 XByteField = 0 (0)  
len 1 ShortField = 60 (60)  
id 1 ShortField = 1111 (1111)  
flags 1 FlagsField = 0 (0)  
frag 1 BitField = 0 (0)  
ttl 1 ByteField = 64 (64)  
proto 1 ShortField = 6 (6)  
checksum 1 XShortField = None (None)  
seq 1 Long = '192.168.56.101' (None)  
window 1 Long = '192.168.56.102' ('192.0.0.1')  
options 1 PacketListField = [] ([])  
sport 1 ShortField = 4444 (4444)  
sport 1 ShortField = [22, 80] (80)  
seq 1 IntField = 12345 (0)  
ack 1 IntField = 1000 (0)  
window 1 BitField = None (None)  
reserved 1 BitField = 0 (0)  
flags 1 FlagsField = 2 (2)  
window 1 ShortField = 1000 (1000)  
checksum 1 XShortField = None (None)  
targets 1 ShortField = 0 (0)  
options 1 TCPOptionsField = 1 (1)  
...  
load 1 BitField = 'HAXOR SVP' (**)  
Sending Packets in 0.3 second intervals for timeout of 6 sec  
RECV 1: IP / TCP 192.168.56.102:8080 > 192.168.56.101:15203 RA / Pending  
IP / TCP 192.168.56.102:8080 > 192.168.56.101:15203 RA / Pending  
RECV 2: IP / TCP 192.168.56.102:8080 > 192.168.56.101:15199 RA / Pending  
IP / TCP 192.168.56.102:8080 > 192.168.56.101:15199 RA / Pending  
RECV 3: IP / TCP 192.168.56.102:8080 > 192.168.56.101:15011 RA / Pending  
IP / TCP 192.168.56.102:8080 > 192.168.56.101:15011 RA / Pending  
RECV 4: IP / TCP 192.168.56.102:8080 > 192.168.56.101:15012 RA / Pending  
IP / TCP 192.168.56.102:8080 > 192.168.56.101:15012 RA / Pending  
RECV 5: IP / TCP 192.168.56.102:8080 > 192.168.56.101:15009 RA / Pending  
IP / TCP 192.168.56.102:8080 > 192.168.56.101:15009 RA / Pending
```

Figure 2: SYN flood script output

```
do  
date >> /var/log/DDOS_IP.log  
netstat | grep -E "ssh|www" | grep -iv ESTABLISHED | awk '{print  
$5}' | cut -d : -f 1 | sort | uniq -c >> /var/log/DDOS_IP.log  
for pip in `netstat | grep -E "ssh|www" | grep -iv ESTABLISHED |  
awk '{print $5}' | cut -d : -f 1 | sort | uniq`  
do  
conntrack=netstat | grep -E "ssh|www" | grep -iv ESTABLISHED |  
awk '{print $5}' | cut -d : -f 1 | grep $pip | wc -l';  
while read line  
do  
if [ "$line" = "$pip" ]  
then  
continue 2  
fi  
done < /tmp/test2.txt  
if [ "$conntrack" -gt "25" ]  
then  
iptables -I INPUT -s $pip -p tcp -j REJECT --reject-with tcp-  
reset  
echo "$pip" >> /tmp/test1.txt  
fi  
done  
cat /tmp/test1.txt | sort | uniq > /tmp/test2.txt  
sleep $1  
done
```

Use it as shown below, on the target server:

```
root@ubiserv:~# ./SYN_Flood_Prevention.sh 4
```